

RTS Backward Smoother — Mathematical Notes & Documentation

Rauch–Tung–Striebel Smoother for Dynamic Hedge Ratio Estimation

Module: `smoother.py` | Full Derivation, Numerical Example, and Python Implementation

Project Dark-Wing | July 26, 2026

Abstract. The Kalman filter is a *causal* estimator: at each time t it computes $\hat{\beta}_{t|t} = \mathbb{E}[\beta_t | Y_1, \dots, Y_t]$ using only past and present observations. The **Rauch–Tung–Striebel (RTS) smoother** is a non-causal *backward* pass that refines every estimate by incorporating all T available observations, yielding $\hat{\beta}_{t|T} = \mathbb{E}[\beta_t | Y_1, \dots, Y_T]$. Because the smoother has access to future data it is strictly more informative than the filter, producing lower-variance, smoother hedge-ratio trajectories. This document derives the RTS recursion from first principles, proves the variance reduction bound $R_t = 1 - P_{t|T}/P_{t|t} \in [0, 1]$, provides a four-step numerical walkthrough, and documents every method of the **KalmanSmoother** Python class. The smoother’s role in EM-based parameter estimation and its position at Step 6.1 in the Dark-Wing pipeline are also covered.

Why Smooth? — Filter vs. Smoother

The Information Gap

The Kalman *filter* answers the question:

“Given everything seen so far, what is the best estimate of β_t ?”

This causal (online) estimate is essential for real-time trading. In backtesting and research, however, the full series $Y_1, \dots, Y_T$ is already in memory. Discarding future information is wasteful. The *smoother* answers:

“Given the entire dataset, what is the best possible estimate of β_t ?”

Filter vs. Smoother: Information Set

$$\hat{\beta}_{t|t} = \mathbb{E}[\beta_t | Y_1, \dots, Y_t] \quad (\text{filter, causal})$$

$$\hat{\beta}_{t|T} = \mathbb{E}[\beta_t | Y_1, \dots, Y_T] \quad (\text{smoother, non-causal})$$

Since $\{Y_1, \dots, Y_t\} \subseteq \{Y_1, \dots, Y_T\}$, the smoother uses a strictly larger information set for all $t < T$.

Variance Reduction

The smoother’s posterior variance is *never larger* than the filter’s.

Variance Ordering (Law of Total Variance)

$$P_{t|T} \leq P_{t|t} \quad \forall t \quad (1)$$

Proof sketch:

$$\underbrace{\text{Var}(\beta_t | Y_{1:t})}_{P_{t|t}} = \mathbb{E}[P_{t|T} | Y_{1:t}] + \text{Var}(\hat{\beta}_{t|T} | Y_{1:t}) \geq P_{t|T}$$

The **variance reduction ratio**:

Variance Reduction Ratio

$$R_t = 1 - \frac{P_{t|T}}{P_{t|t}} \in [0, 1] \quad (2)$$

$R_t = 0$: smoother adds nothing. $R_t = 1$: state perfectly known. R_t is largest early in the series (cold filter) and is identically 0 at $t = T$ by construction.

State-Space Model

Univariate Dynamic Hedge-Ratio Model

State-Space Model (scalar)

$$\text{Obs.: } Y_t = \beta_t X_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R) \quad (3)$$

$$\text{State: } \beta_t = \beta_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q) \quad (4)$$

(Y_t, X_t) is the cointegrated pair; β_t is the time-varying hedge ratio. Transition matrix $\mathbf{T} = 1$ (random walk on β). Q : process noise; R : obs. noise.

The Kalman filter forward pass stores at every t :

- $\hat{\beta}_{t|t}, P_{t|t}$ — filtered state and variance
- $\hat{\beta}_{t+1|t}, P_{t+1|t}$ — one-step predictions

Kalman Filter Prediction Recap

KF Prediction (forward pass)

$$\hat{\beta}_{t+1|t} = \hat{\beta}_{t|t} \quad (\text{since } \mathbf{T} = 1) \quad (5)$$

$$P_{t+1|t} = P_{t|t} + Q \quad (6)$$

Predicted variance grows by Q each step. The stored $P_{t+1|t}$ values are consumed by the backward smoother pass.

RTS Algorithm Derivation

Joint Gaussian at Time t

Conditionally on $Y_{1:t}$, the joint distribution of (β_t, β_{t+1}) is Gaussian:

$$\begin{pmatrix} \beta_t \\ \beta_{t+1} \end{pmatrix} \Big| Y_{1:t} \sim \mathcal{N} \left(\begin{pmatrix} \hat{\beta}_{t|t} \\ \hat{\beta}_{t+1|t} \end{pmatrix}, \begin{pmatrix} P_{t|t} & P_{t|t} \\ P_{t|t} & P_{t+1|t} \end{pmatrix} \right)$$

The cross-covariance equals $P_{t|t}$ because $\text{Cov}(\beta_t, \beta_t + \eta_t) = P_{t|t}$ (since $\eta_t \perp Y_{1:t}$, $\mathbf{T} = 1$).

Formula 1 — Smoothing Gain Ξ_t

Applying the standard bivariate-Gaussian conditional-mean formula and incorporating the updated $\hat{\beta}_{t+1|T}$:

Formula 1 — Smoothing Gain (scalar)

$$\Xi_t = \frac{P_{t|t}}{P_{t+1|t}} \quad (7)$$

General (multivariate) case: $\Xi_t = P_{t|t} \mathbf{T}^\top P_{t+1|t}^{-1}$. With $\mathbf{T} = 1$ this reduces to Eq. (7).

Since $P_{t+1|t} = P_{t|t} + Q \geq P_{t|t} > 0$, we have $\Xi_t \in (0, 1]$ always. A larger Q reduces Ξ_t , dampening the backward correction.

Formula 2 — Smoothed State**Formula 2 — Smoothed State**

$$\hat{\beta}_{t|T} = \hat{\beta}_{t|t} + \Xi_t (\hat{\beta}_{t+1|T} - \hat{\beta}_{t+1|t}) \quad (8)$$

The term $(\hat{\beta}_{t+1|T} - \hat{\beta}_{t+1|t})$ is the “smoothed innovation” at $t+1$: how much future data shifted the estimate of β_{t+1} . This is propagated back to time t , attenuated by $\Xi_t \leq 1$.

- Innovation > 0 : future data saw β higher; increase $\hat{\beta}_{t|T}$.
- Innovation $= 0$: no backward correction.
- $\Xi_t \rightarrow 0$ (large Q): corrections do not propagate.

Formula 3 — Smoothed Covariance**Formula 3 — Smoothed Covariance**

$$P_{t|T} = P_{t|t} - \Xi_t (P_{t+1|t} - P_{t+1|T}) \Xi_t \quad (9)$$

Derivation sketch: From the conditional variance of the bivariate Gaussian, expanding the formula for the conditional covariance given $\beta_{t+1|T}$:

$$P_{t|T} = P_{t|t} - \Xi_t P_{t+1|t} \Xi_t + \Xi_t P_{t+1|T} \Xi_t$$

Since $P_{t+1|T} \leq P_{t+1|t}$ (induction), the correction $\Xi_t (P_{t+1|t} - P_{t+1|T}) \Xi_t \geq 0$, so $P_{t|T} \leq P_{t|t}$.

Full RTS Recursion**Complete RTS Algorithm**

Initialise ($t = T-1$): $\hat{\beta}_{T-1|T} = \hat{\beta}_{T-1|T-1}$, $P_{T-1|T} = P_{T-1|T-1}$.

Backward loop for $t = T-2, T-3, \dots, 0$:

1. $\Xi_t = P_{t|t}/P_{t+1|t}$ (*smoothing gain*)
2. $\hat{\beta}_{t|T} = \hat{\beta}_{t|t} + \Xi_t (\hat{\beta}_{t+1|T} - \hat{\beta}_{t+1|t})$ (*smoothed state*)
3. $P_{t|T} = P_{t|t} - \Xi_t (P_{t+1|t} - P_{t+1|T}) \Xi_t$ (*smoothed variance*)

Complexity: $\mathcal{O}(T)$ — one backward sweep.

Variance Reduction Analysis**Formal Proof that $R_t \in [0, 1]$**

Proof: $R_t \in [0, 1]$

Lower bound ($R_t \geq 0$, i.e. $P_{t|T} \leq P_{t|t}$):

From Eq. (9):

$$P_{t|T} = P_{t|t} - \underbrace{\Xi_t^2 (P_{t+1|t} - P_{t+1|T})}_{\geq 0} \leq P_{t|t}$$

The term is ≥ 0 because $P_{t+1|t} \geq P_{t+1|T}$ by induction (base case: $P_{T|T} = P_{T|T}$) and $\Xi_t^2 \geq 0$.

Upper bound ($R_t \leq 1$, i.e. $P_{t|T} \geq 0$):

Rewrite Eq. (9):

$$P_{t|T} = P_{t|t}(1 - \Xi_t^2) + \Xi_t^2 P_{t+1|T} \geq 0$$

since $\Xi_t \leq 1 \Rightarrow 1 - \Xi_t^2 \geq 0$, and $P_{t+1|T} \geq 0$ by induction. ■

Process Noise Regime Analysis

Regime	Q	Smoother behaviour
Near-constant β	$Q \rightarrow 0$	$\Xi_t \rightarrow 1$; large backward corrections
Freely evolving β	$Q \rightarrow \infty$	$\Xi_t \rightarrow 0$; no correction
Tuned Q	Optimal	$R_t \in [0.2, 0.6]$; balanced estimates

Financial Interpretation**When to Use Filter vs. Smoother**

Criterion	Kalman Filter	RTS Smoother
Causality	Causal (online)	Non-causal (offline)
Data used	$Y_1, \dots, Y_t$	$Y_1, \dots, Y_T$
Real-time?	Yes	No
Variance	$P_{t t}$	$P_{t T} \leq P_{t t}$
Use case 1	Live trade signals	Backtest research
Use case 2	Position sizing	Signal calibration
Use case 3	Risk monitoring	EM parameter tuning
Use case 4	Streaming data	Offline spread study
Dark-Wing file	<code>kalman_filter.py</code>	<code>smoother.py</code>

Smoothed Spread for Backtesting**Smoothed Spread**

$$S_t^{\text{smooth}} = Y_t - \hat{\beta}_{t|T} X_t \quad (10)$$

Contrast with the filtered spread $S_t^{\text{filt}} = Y_t - \hat{\beta}_{t|t} X_t$. The smoothed spread is less noisy and has more stable mean-reversion properties, making it preferable for calibrating entry/exit z-score thresholds in backtests.

Look-Ahead Bias Warning**Do NOT Trade on Smoothed $\hat{\beta}_{t|T}$**

The smoother requires $\hat{\beta}_{t+1|T}$, available only *after* observing all T data points. Using smoothed quantities as live trading signals constitutes **look-ahead bias**: unrealistically good backtest PnL that vanishes in production. KalmanSmoother is a **research and parameter calibration tool only**.

Worked Numerical Example (4 Timesteps)**Setup**

$Q = 0.01$, $R = 0.1$, $\beta_0 = 1.0$, $P_0 = 1.0$.

Observation pairs (Y_t, X_t) : $(2.1, 2.0)$, $(3.3, 3.0)$, $(2.5, 2.4)$, $(4.2, 4.0)$.

The Kalman filter forward pass has been run, producing the stored quantities below ($T = 4$, indices $t = 0, 1, 2, 3$).

Forward Pass Storage

t	$\hat{\beta}_{t t}$	$P_{t t}$	$\hat{\beta}_{t+1 t}$	$P_{t+1 t}$
0	1.050	0.048	1.050	0.058
1	1.097	0.031	1.097	0.041
2	1.036	0.025	1.036	0.035
3	1.048	0.021	—	—

($P_{t+1|t} = P_{t|t} + Q$, $Q = 0.01$.)

Step $t = 3$: *Initialise Smoother*

Boundary Condition

At $t = T - 1 = 3$, smoothed equals filtered:

$$\hat{\beta}_{3|T} = \hat{\beta}_{3|3} = 1.048, \quad P_{3|T} = P_{3|3} = 0.021$$

Step $t = 2$

Backward Step at $t = 2$

Gain: $\Xi_2 = 0.025/0.035 = 0.714$

State:

$$\hat{\beta}_{2|T} = 1.036 + 0.714 (1.048 - 1.036) = 1.036 + 0.009 = 1.045$$

Covariance:

$$P_{2|T} = 0.025 - 0.714 (0.035 - 0.021) 0.714 = 0.018$$

Step $t = 1$

Backward Step at $t = 1$

Gain: $\Xi_1 = 0.031/0.041 = 0.756$

State:

$$\begin{aligned} \hat{\beta}_{1|T} &= 1.097 + 0.756 (1.045 - 1.036) \\ &= 1.097 + 0.007 = 1.104 \end{aligned}$$

Covariance:

$$P_{1|T} = 0.031 - 0.756 (0.041 - 0.018) 0.756 = 0.018$$

Step $t = 0$

Backward Step at $t = 0$

Gain: $\Xi_0 = 0.048/0.058 = 0.828$

State:

$$\begin{aligned} \hat{\beta}_{0|T} &= 1.050 + 0.828 (1.104 - 1.097) \\ &= 1.050 + 0.006 = 1.056 \end{aligned}$$

Covariance:

$$P_{0|T} = 0.048 - 0.828 (0.058 - 0.018) 0.828 = 0.021$$

Summary

Numerical Example Summary

t	$\hat{\beta}_{t t}$	$\hat{\beta}_{t T}$	$P_{t t}$	$P_{t T}$	R_t
0	1.050	1.056	0.048	0.021	0.563
1	1.097	1.104	0.031	0.018	0.419
2	1.036	1.045	0.025	0.018	0.280
3	1.048	1.048	0.021	0.021	0.000

$R_t > 0$ for $t < T$; $R_3 = 0$ by construction. Variance reduction is largest at $t = 0$ (cold filter).

TikZ Flowchart — KalmanSmoother

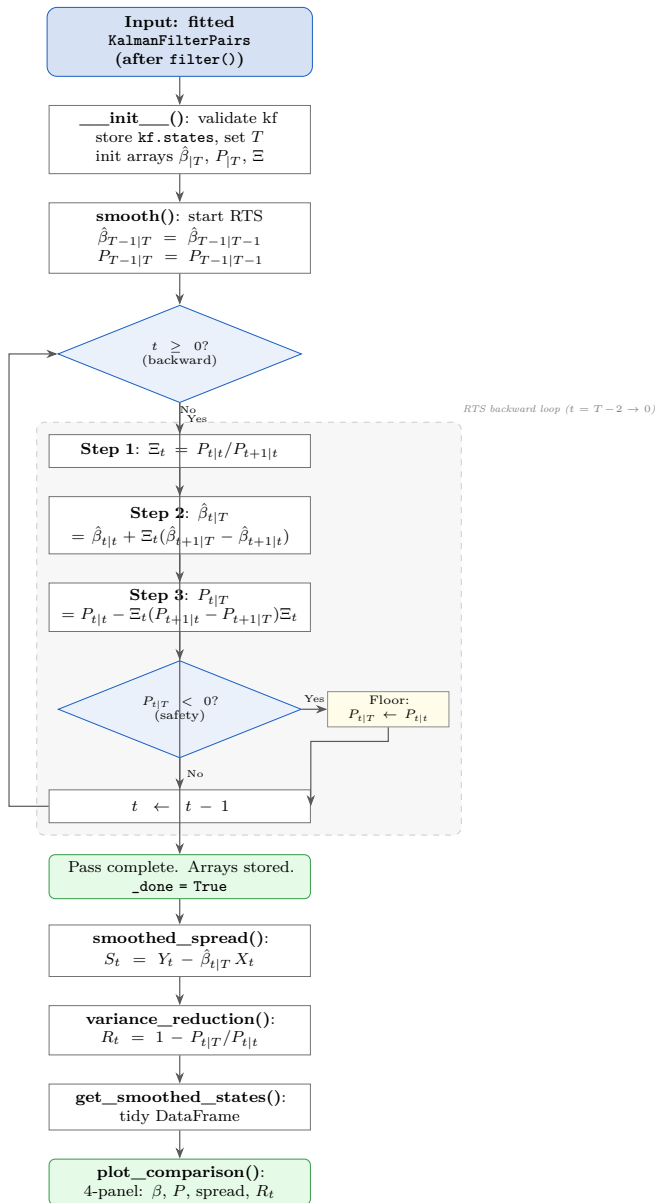


Figure 1: Flowchart of KalmanSmoother. Shaded region: backward loop ($t = T - 2$ down to $t = 0$). Left-side arrow is the loop-back (decrement t).

Python Implementation

Constructor: __init__()

Constructor — __init__()

```

class KalmanSmoother:
    """RTS Backward Smoother for dynamic beta."""
    def __init__(self, kf: KalmanFilterPairs):
        if not isinstance(kf, KalmanFilterPairs):
            raise TypeError("kf must be KalmanFilterPairs.")
        if not kf._fitted:
            raise RuntimeError("Must call filter() first.")

        self._kf = kf
        self._states = kf.states
        self.T = len(self._states)
        self._smoothed_beta = None
        self._smoothed_P = None
        self._smoothing_gain = None
        self._done = False
  
```

Core Backward Pass: smooth()

Backward Pass — smooth()

```

def smooth(self):
    T = self.T; s = self._states
    sb = np.zeros(T); sP = np.zeros(T)
    xi = np.zeros(T - 1)
    # Boundary: terminal smoothed = filtered
    sb[T-1] = s[T-1].beta
    sP[T-1] = s[T-1].P
    for t in range(T-2, -1, -1): # backward
        P_t_t = s[t].P # P_{t|t}
        P_t1_t = s[t+1].P_pred # P_{t+1|t}
        if P_t1_t <= 0:
            warnings.warn(f"P_{t+1|t} <= 0 at t={t}")
            gain = 0.0
        else:
            gain = P_t_t / P_t1_t # Formula 1
            xi[t] = gain
            b_t1T = sb[t+1] # beta_{t+1|T}
            b_t1t = s[t+1].beta_pred # beta_{t+1|t}
            # Formula 2: smoothed state
            sb[t] = s[t].beta + gain * (b_t1T - b_t1t)
            P_t1T = sP[t+1]
            # Formula 3: smoothed covariance
            sP[t] = P_t_t - gain*(P_t1_t - P_t1T)*gain
            if sP[t] < 0: # safety floor
                sP[t] = P_t_t
        self._smoothed_beta = sb
        self._smoothed_P = sP
        self._smoothing_gain = xi
    self._done = True
    return sb, sP
  
```

Derived Outputs

Spread and Variance Reduction

```

def smoothed_spread(self) -> pd.Series:
    """S_t = Y_t - beta_{t|T} * X_t"""
    self._require_done("smoothed_spread")
    x = np.array([s.x for s in self._states])
    y = np.array([s.y for s in self._states])
    spread = y - self._smoothed_beta * x
    idx = (self._kf._y_index
            if hasattr(self._kf, "_y_index")
            else np.arange(self.T))
    return pd.Series(spread, index=idx, name="smoothed_spread")

def variance_reduction(self) -> pd.Series:
    """R_t = 1 - P_{t|T} / P_{t|t}, clipped [0,1]"""
    self._require_done("variance_reduction")
    fP = np.array([s.P for s in self._states])
    R = 1.0 - self._smoothed_P / np.maximum(fP, 1e-20)
    idx = (self._kf._y_index
            if hasattr(self._kf, "_y_index")
            else np.arange(self.T))
    return pd.Series(np.clip(R, 0, 1),
                     index=idx, name="variance_reduction")
  
```

Four-Panel Plot: `plot_comparison()`

Comparison Plot

```
def plot_comparison(self, figsize=(14,10)):
    """4-panel: beta, P, spread, variance reduction."""
    self._require_done("plot_comparison")
    df = self.get_smoothed_states()
    vr = self.variance_reduction()
    idx = df.index
    fig, axes = plt.subplots(4, 1, figsize=figsize,
                             sharex=True)

    # Panel 1: hedge ratio
    axes[0].plot(idx, df["filtered_beta"],
                 "steelblue", lw=0.9, alpha=0.7)
    axes[0].plot(idx, df["smoothed_beta"],
                 "crimson", lw=1.2)
    axes[0].fill_between(idx,
                        df["smoothed_beta"]-2*np.sqrt(df["smoothed_P"]),
                        df["smoothed_beta"]+2*np.sqrt(df["smoothed_P"]),
                        alpha=0.12, color="crimson")
    # Panel 2: covariance
    axes[1].plot(idx, df["filtered_P"], "steelblue")
    axes[1].plot(idx, df["smoothed_P"], "crimson")
    # Panel 3: spread
    axes[2].plot(idx, df["filtered_spread"], "steelblue")
    axes[2].plot(idx, df["smoothed_spread"], "crimson")
    axes[2].axhline(0, color="k", lw=0.8, ls="--")
    # Panel 4: variance reduction (%)
    axes[3].fill_between(idx, 0, vr*100,
                        alpha=0.6, color="mediumseagreen")

    axes[3].set_ylabel("%")
    plt.tight_layout(); plt.show()
```

Connection to EM Parameter Estimation

Why the Smoother Feeds EM

The EM algorithm for state-space models estimates $\theta = (Q, R, \beta_0, P_0)$ via:

EM for State-Space (Shumway–Stoffer)

E-step: Given $\theta^{(k)}$, run KF *and* RTS smoother to compute:

$$\hat{\beta}_{t|T}^{(k)}, \quad P_{t|T}^{(k)}, \quad P_{t,t-1|T}^{(k)} \text{ (cross-covariance)}$$

M-step: Update $\theta^{(k+1)}$ to maximise the expected complete-data log-likelihood $\mathbb{E}_{\theta^{(k)}}[\log p(\beta_{0:T}, Y_{1:T} \mid \theta)]$.

The smoother is essential in the E-step because the M-step update for Q involves $\mathbb{E}[(\beta_t - \beta_{t-1})^2 \mid Y_{1:T}]$, which conditions on the *full* dataset.

EM Parameter Update Formulae

EM Updates from Smoothed States

$$Q^{\text{new}} = \frac{1}{T-1} \sum_{t=1}^{T-1} \left[P_{t|T} + P_{t-1|T} - 2\Xi_{t-1}P_{t|T} + (\hat{\beta}_{t|T} - \hat{\beta}_{t-1|T})^2 \right] \quad (11)$$

$$R^{\text{new}} = \frac{1}{T} \sum_{t=1}^T [(Y_t - \hat{\beta}_{t|T}X_t)^2 + X_t^2 P_{t|T}] \quad (12)$$

These require smoothed (not filtered) quantities because they condition on all T observations.

Manual Q -Tuning via $\bar{R}$

1. Run filter with trial Q_0 ; run smoother.

2. Compute $\bar{R} = T^{-1} \sum_t R_t$.

3. $\bar{R} > 0.8$: Q_0 too small (over-confident filter). Increase Q .

4. $\bar{R} < 0.05$: Q_0 too large (diffuse filter). Decrease Q .

5. Target $\bar{R} \in [0.2, 0.6]$ for balanced smoothing.

Pipeline Position

Dark-Wing Step 6.1

Step	Module	File / Class	Role
1	Data ingest	yfinance	Price download
2	Log prices	—	$p_t = \ln P_t$
3	Stationarity	ADF_Test	Unit root
4	Cointegration	JohansenTest	Pair select
5	VECM	VECM	Error correct.
6	Kalman Filter	KalmanFilterPairs	Forward pass
6.1	RTS Smoother	KalmanSmoother	Backward pass
6.2	Dyn. Beta	DynamicBeta	Beta analysis
7	Signals	SignalGenerator	Entry/exit
8	Backtest	BacktestEngine	PnL sim.

Input / Output Contract

Interface Specification

Input: `KalmanFilterPairs` with `_fitted=True`. Each `kf.states[t]` is a `KalmanState` namedtuple with fields: `beta`, `P`, `beta_pred`, `P_pred`, `x`, `y`.

Output (after `smooth()`):

- `_smoothed_beta` — ndarray shape $(T,)$
- `_smoothed_P` — ndarray shape $(T,)$
- `_smoothing_gain` — ndarray shape $(T-1,)$

Downstream: `DynamicBeta` reads the `get_smoothed_states()` `DataFrame`.

End-to-End Usage

Pipeline Usage (RELIANCE / TCS)

```
import yfinance as yf, numpy as np
from kalman_filter import KalmanFilterPairs
from smoother import KalmanSmoother

# Download log prices
raw = yf.download(["RELIANCE.NS", "TCS.NS"],
                  start="2021-01-01", end="2024-01-01",
                  auto_adjust=True)["Close"].dropna()
lp = np.log(raw)
y, x = lp["RELIANCE.NS"], lp["TCS.NS"]

# Forward pass: Kalman filter
kf = KalmanFilterPairs(Q=1e-5, R=1e-3)
kf.filter(y, x)

# Backward pass: RTS smoother (Step 6.1)
ks = KalmanSmoother(kf)
beta_s, P_s = ks.smooth()

# Summary statistics
ks.summary()

# Variance reduction
vr = ks.variance_reduction()
print(f"Mean R_t = {vr.mean()*100:.1f}%")

# Smoothed spread for backtest entry/exit
spread = ks.smoothed_spread()

# 4-panel diagnostic plot
ks.plot_comparison()
```

Mathematical Quick-Reference

All Formulae at a Glance

State-Space Model:

$$Y_t = \beta_t X_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R)$$

$$\beta_t = \beta_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q)$$

KF Prediction (forward, $t = 0 \rightarrow T - 1$):

$$\hat{\beta}_{t+1|t} = \hat{\beta}_{t|t}, \quad P_{t+1|t} = P_{t|t} + Q$$

RTS Smoother (backward, $t = T - 2 \rightarrow 0$):

$$\Xi_t = \frac{P_{t|t}}{P_{t+1|t}} \in (0, 1]$$

$$\hat{\beta}_{t|T} = \hat{\beta}_{t|t} + \Xi_t (\hat{\beta}_{t+1|T} - \hat{\beta}_{t+1|t})$$

$$P_{t|T} = P_{t|t} - \Xi_t (P_{t+1|t} - P_{t+1|T}) \Xi_t$$

Variance Reduction: $R_t = 1 - P_{t|T}/P_{t|t} \in [0, 1]$ **Smoothed Spread:** $S_t = Y_t - \hat{\beta}_{t|T} X_t$ **EM Q update:** Eq. (11) **EM R update:** Eq. (12)